

B+ tree

A **B+ tree** is an m-ary tree with a variable but often large number of children per node. A B+ tree consists of a root, internal nodes, and leaves.^[1] The root may be either a leaf or a node with two or more children.

A B+ tree can be viewed as a B-tree in which each node contains only keys (not key–value pairs), and to which an additional level is added at the bottom with linked leaves.

The primary value of a B+ tree is in storing data for efficient retrieval in a block-oriented storage context—in particular, filesystems. This is primarily because unlike binary search trees, B+ trees have very high fanout (number of pointers to child nodes in a node,^[1] typically on the order of 100 or more), which reduces the number of I/O operations required to find an element in the tree.

B+ tree		
<u>Type</u>	<u>Tree (data structure)</u>	
<u>Time complexity in big O notation</u>		
Operation	Average	Worst case
Search	$O(\log n)$	$O(\log n)$
Insert	$O(\log n)$	$O(\log n)$
Delete	$O(\log n)$	$O(\log n)$
<u>Space complexity</u>		
Space	$O(n)$	$O(n)$

History

There is no single paper introducing the B+ tree concept. Instead, the notion of maintaining all data in leaf nodes is repeatedly brought up as an interesting variant of the B-tree, which was introduced by R. Bayer and E. McCreight.^[2] Douglas Comer notes in an early survey of B-trees (which also covers B+ trees) that the B+ tree was used in IBM's VSAM data access software, and refers to an IBM published article from 1973.^[3]

Structure

Pointer structure

As with other trees, B+ trees can be represented as a collection of three types of nodes: *root*, *internal* (a.k.a. interior), and *leaf*. In B+ trees, the following properties are maintained for these nodes:

- If k_i exists in any node in a B+ tree, then k_{i-1} exists in that node where $i \geq 1$.
- All leaf nodes have the same number of ancestors (i.e., they are all at the same depth).

The pointer properties of nodes are summarized in the tables below:

- K : Maximum number of potential search keys for each node in a B+ tree. (this value is constant over the entire tree).
- p_i : The pointer at the zero-based node index i .
- k_i : The search key at the zero-based node index i .

p_0		p_1		p_2		p_3		p_4
	k_0		k_1		k_2		k_3	

B+ tree node format where $K=4$. (p_i represents the pointers, k_i represents the search keys).

Internal Node Pointer Structure

	p_0	p_i		
when	k_0 exists	k_{i-1} and k_i exist	k_{i-1} exists, and k_i does not exist	k_{i-1} and k_i do not exist
P: Points to subtree in which all search keys	$P < k_0$.	$k_{i-1} \leq P < k_i$.	$P \geq k_{i-1}$.	p_i is empty.

Leaf Node Pointer Structure

p_i when k_i exists	p_i when k_i does not exist and $i \neq K$	p_K
Points to a record with a value equal to k_i .	Here, p_i is empty.	Points to the next leaf in the tree.

Node bounds

The node bounds are summarized in the table below:^{[4][5]}

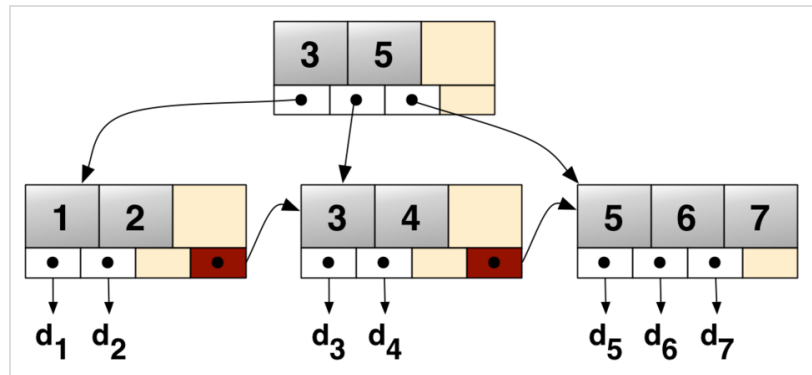
Node Type	Number of Keys		Number of Child Nodes	
	Min	Max	Min	Max
Root Node (when it is a leaf node)	0	K	0	0
Root Node (when it is an internal node)	1	K	$2^{\lceil 1 \rceil}$	$K + 1$
Internal Node	$\lfloor K/2 \rfloor$	K	$\lceil (K + 1)/2 \rceil \equiv \lfloor K/2 \rfloor + 1$	$K + 1$
Leaf Node	$\lceil K/2 \rceil$	K	0	0

Intervals in internal nodes

By definition, each value contained within the B+ tree is a key contained in exactly one leaf node. Each key is required to be directly comparable with every other key, which forms a total order.^[6] This enables each leaf node to keep all of its keys sorted at all times, which then enables each internal node to construct an ordered collection of intervals representing the contiguous extent

of values contained in a given leaf. Internal nodes higher in the tree can then construct their own intervals, which recursively aggregate the intervals contained in their own child internal nodes. Eventually, the root of a B+ Tree represents the whole range of values in the tree, where every internal node represents a subinterval.

For this recursive interval information to be retained, internal nodes must additionally contain $m - 1$ copies of keys l_i for $i \in [1, m - 1]$ representing the least element within the interval covered by the child with index i (which may itself be an internal node, or a leaf). Where m represents the *actual* number of children for a given internal node.



A simple B+ tree example linking the keys 1–7 to data values d_1 – d_7 . The linked list (red) allows rapid in-order traversal. This particular tree's branching factor is $b=4$. Both keys in leaf and internal nodes are colored gray here.

Characteristics

The *order* or *branching factor* b of a B+ tree measures the capacity of interior nodes, i.e. their maximum allowed number of direct child nodes. This value is constant over the entire tree. For a b -order B+ tree with h levels of index:

- The maximum number of records stored is $n_{\max} = b^h - b^{h-1}$ (b^{h-1} is subtracted to account for the next pointers which do not point to data records but instead point to the next leaf node)
- The minimum number of records stored is $n_{\min} = 2 \left\lceil \frac{b}{2} \right\rceil^{h-1} - 2 \left\lceil \frac{b}{2} \right\rceil^{h-2}$
- The minimum number of keys is $n_{\text{kmin}} = 2 \left\lceil \frac{b}{2} \right\rceil^{h-1} - 1$
- The maximum number of keys is $n_{\text{kmax}} = b^h - 1$
- The space required to store the tree is $O(n)$
- Inserting a record requires $O(\log_b n)$ operations
- Finding a record requires $O(\log_b n)$ operations
- Removing a (previously located) record requires $O(\log_b n)$ operations
- Performing a range query with k elements occurring within the range requires $O(\log_b n + k)$ operations
- The B+ tree structure expands/contracts as the number of records increases/decreases. There are no restrictions on the size of B+ trees. Thus, increasing usability of a database

system.

- Any change in structure does not affect performance due to balanced tree properties.^[7]
- The data is stored in the leaf nodes and more branching of internal nodes helps to reduce the tree's height, thus, reduce search time. As a result, it works well in secondary storage devices.^[8]
- Searching becomes extremely simple because all records are stored only in the leaf node and are sorted sequentially in the linked list.
- We can retrieve range retrieval or partial retrieval using B+ tree. This is made easier and faster by traversing the tree structure. This feature makes B+ tree structure applied in many search methods.^[7]

Algorithms

Search

We are looking for a value k in the B+ Tree. This means that starting from the root, we are looking for the leaf which may contain the value k . At each node, we figure out which internal node we should follow. An internal B+ Tree node has at most $m \leq b$ children, where every one of them represents a different sub-interval. We select the corresponding child via a linear search of the m entries, then when we finally get to a leaf, we do a linear search of its n elements for the desired key. Because we only traverse one branch of all the children at each rung of the tree, we achieve $O(\log N)$ runtime, where N is the total number of keys stored in the leaves of the B+ tree.^[4]

```
function search(k, root) is
    let leaf = leaf_search(k, root)
    for leaf_key in leaf.keys():
        if k = leaf_key:
            return true
    return false
```

```
function leaf_search(k, node) is
    if node is a leaf:
        return node
    let p = node.children()
    let l = node.left_sided_intervals()
    assert |p| = |l| + 1
    let m = p.len()
    for i from 1 to m - 1:
        if k ≤ l[i]:
            return leaf_search(k, p[i])
    return leaf_search(k, p[m])
```

Note that this pseudocode uses 1-based array indexing.

Insertion

- Perform a search to determine which node the new record should go into.

- If the node is not full (at most $b - 1$ entries after the insertion), add the record.
- Otherwise, *before* inserting the new record
 - Split the node.
 - original node has $\lceil (K + 1)/2 \rceil$ items
 - new node has $\lfloor (K + 1)/2 \rfloor$ items
 - Copy $\lceil (K + 1)/2 \rceil$ -th key to the parent, and insert the new node to the parent.
 - Repeat until a parent is found that need not split.
 - Insert the new record into the new node.
- If the root splits, treat it as if it has an empty parent and split as outlined above.

B+ trees grow at the root and not at the leaves.^[1]

Bulk-loading

Given a collection of data records, we want to create a B+ tree index on some key field. One approach is to insert each record into an empty tree. However, it is quite expensive, because each entry requires us to start from the root and go down to the appropriate leaf page. An efficient alternative is to use bulk-loading.

- The first step is to sort the data entries according to a search key in ascending order.
- We allocate an empty page to serve as the root, and insert a pointer to the first page of entries into it.
- When the root is full, we split the root, and create a new root page.
- Keep inserting entries to the right most index page just above the leaf level, until all entries are indexed.

Note:

- when the right-most index page above the leaf level fills up, it is split;
- this action may, in turn, cause a split of the right-most index page one step closer to the root;
- splits only occur on the right-most path from the root to the leaf level.^[9]

Deletion

The purpose of the delete algorithm is to remove the desired entry node from the tree structure. We recursively call the delete algorithm on the appropriate node until no node is found. For each function call, we traverse along, using the index to navigate until we find the node, remove it, and then work back up to the root.

At entry L that we wish to remove:

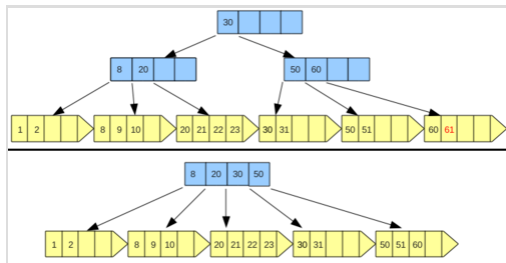
- If L is at least half-full, done
- If L has only d-1 entries, try to re-distribute, borrowing from sibling (adjacent node with same

parent as L).

After the re-distribution of two sibling nodes happens, the parent node must be updated to reflect this change. The index key that points to the second sibling must take the smallest value of that node to be the index key.

- If re-distribute fails, merge L and sibling. After merging, the parent node is updated by deleting the index key that point to the deleted entry. In other words, if merge occurred, must delete entry (pointing to L or sibling) from parent of L.

Note: merge could propagate to root, which means decreasing height.^[10]



B+ tree deletion

Implementation

The leaves (the bottom-most index blocks) of the B+ tree are often linked to one another in a linked list; this makes range queries or an (ordered) iteration through the blocks simpler and more efficient (though the aforementioned upper bound can be achieved even without this addition). This does not substantially increase space consumption or maintenance on the tree. This illustrates one of the significant advantages of a B+tree over a B-tree; in a B-tree, since not all keys are present in the leaves, such an ordered linked list cannot be constructed. A B+tree is thus particularly useful as a database system index, where the data typically resides on disk, as it allows the B+tree to actually provide an efficient structure for housing the data itself (this is described in^{[11]:238} as index structure "Alternative 1").

If a storage system has a block size of B bytes, and the keys to be stored have a size of k , arguably the most efficient B+ tree is one where $b = \frac{B}{k} - 1$. Although theoretically the one-off is unnecessary, in practice there is often a little extra space taken up by the index blocks (for example, the linked list references in the leaf blocks). Having an index block which is slightly larger than the storage system's actual block represents a significant performance decrease; therefore erring on the side of caution is preferable.

If nodes of the B+ tree are organized as arrays of elements, then it may take a considerable time to insert or delete an element as half of the array will need to be shifted on average. To overcome this problem, elements inside a node can be organized in a binary tree or a B+ tree instead of an array.

B+ trees can also be used for data stored in RAM. In this case a reasonable choice for block size would be the size of processor's cache line.

Space efficiency of B+ trees can be improved by using some compression techniques. One possibility is to use delta encoding to compress keys stored into each block. For internal blocks, space saving can be achieved by either compressing keys or pointers. For string keys, space can be saved by using the following technique: Normally the i -th entry of an internal block contains the first key of block $i + 1$. Instead of storing the full key, we could store the shortest prefix of the first key of block $i + 1$ that is strictly greater (in lexicographic order) than last key of block i . There is also a simple way to compress pointers: if we suppose that some consecutive blocks $i, i + 1, \dots, i + k$ are stored contiguously, then it will suffice to store only a pointer to the first block and the count of consecutive blocks.

All the above compression techniques have some drawbacks. First, a full block must be decompressed to extract a single element. One technique to overcome this problem is to divide each block into sub-blocks and compress them separately. In this case searching or inserting an element will only need to decompress or compress a sub-block instead of a full block. Another drawback of compression techniques is that the number of stored elements may vary considerably from a block to another depending on how well the elements are compressed inside each block.

Applications

File systems

The ReiserFS, NSS, XFS, JFS, ReFS, and BFS filesystems all use this type of tree for metadata indexing; BFS also uses B+ trees for storing directories. NTFS uses B+ trees for directory and security-related metadata indexing. EXT4 uses extent trees (a modified B+ tree data structure) for file extent indexing.^[12] APFS uses B+ trees to store mappings from filesystem object IDs to their locations on disk, and to store filesystem records (including directories), though these trees' leaf nodes lack sibling pointers.^[13]

Database systems

Relational database management systems such as IBM Db2,^[11] Informix,^[11] Microsoft SQL Server,^[11] Oracle 8,^[11] Sybase ASE,^[11] and SQLite^[14] support this type of tree for table indices, though each such system implements the basic B+ tree structure with variations and extensions. Many NoSQL database management systems such as CouchDB^{[15][a]} and Tokyo Cabinet^[16] also support this type of tree for data access and storage.

Finding objects in a high-dimensional database that are comparable to a particular query object is one of the most often utilized and yet expensive procedures in such systems. In such situations, finding the closest neighbor using a B+ tree is productive.^[17]

iDistance

B+ tree is efficiently used to construct an indexed search method called iDistance. iDistance

searches for k nearest neighbors (kNN) in high-dimension metric spaces. The data in those high-dimension spaces is divided based on space or partition strategies, and each partition has an index value that is close with the respect to the partition. From here, those points can be efficiently implemented using B+ tree, thus, the queries are mapped to single dimensions ranged search. In other words, the iDistance technique can be viewed as a way of accelerating the sequential scan. Instead of scanning records from the beginning to the end of the data file, the iDistance starts the scan from spots where the nearest neighbors can be obtained early with a very high probability.^[18]

NVRAM

Nonvolatile random-access memory (NVRAM) has been using B+ tree structure as the main memory access technique for the Internet Of Things (IoT) system because of its non static power consumption and high solidity of cell memory. B+ can regulate the trafficking of data to memory efficiently. Moreover, with advanced strategies on frequencies of some highly used leaf or reference point, the B+ tree shows significant results in increasing the endurance of database systems.^[19]

See also

- [Binary search tree](#)
- [B-tree](#)
- [Divide-and-conquer algorithm](#)

Notes

- See note after 3rd paragraph.

References

1. [Elmasri, Ramez](#); [Navathe, Shamkant B.](#) (2010). *Fundamentals of database systems* (6th ed.). Upper Saddle River, N.J.: Pearson Education. pp. 652–660. ISBN 9780136086208. LCCN 2010010677 (<https://lccn.loc.gov/2010010677>). OCLC 586123196 (<https://search.worldcat.org/oclc/586123196>). OL 24411294M (<https://openlibrary.org/books/OL24411294M>).
2. Bayer, R.; McCreight, E. (November 1970). "Organization and maintenance of large ordered indices" (<https://dl.acm.org/doi/abs/10.1145/1734663.1734671>). *Proceedings of the 1970 ACM SIGFIDET (Now SIGMOD) Workshop on Data Description, Access and Control - SIGFIDET '70*. pp. 107–141. doi:10.1145/1734663.1734671 (<https://doi.org/10.1145%2F1734663.1734671>). ISBN 978-1-4503-7941-0.

3. Comer, Douglas (1979). "Ubiquitous B-Tree" (<https://doi.org/10.1145%2F356770.356776>). *ACM Computing Surveys*. **11** (2): 121–137. doi:10.1145/356770.356776 (<https://doi.org/10.1145%2F356770.356776>). S2CID 101673 (<https://api.semanticscholar.org/CorpusID:101673>).
4. Pollari-Malmi, Kerttu. " "B+ trees" " (<https://web.archive.org/web/20210414050947/https://www.cs.helsinki.fi/u/mluukkai/tirak2010/B-tree.pdf>) (PDF). *Computer Science, Faculty of Science, University of Helsinki*. p. 3. Archived from the original (<https://www.cs.helsinki.fi/u/mluukkai/tirak2010/B-tree.pdf>) (PDF) on 14 April 2021.
5. Silberschatz, Abraham; Korth, Henry F.; Sudarshan, S. (2020). *Database system concepts* (Seventh ed.). New York, NY: McGraw-Hill Education. ISBN 978-1-260-08450-4.
6. Grust, Torsten (Summer 2013). " "Tree-Structured Indexing: ISAM and B+-trees" " (<https://web.archive.org/web/20201031195459/https://db.inf.uni-tuebingen.de/staticfiles/teaching/ss13/db2/db2-04-1up.pdf>) (PDF). *Logo der Universität Tübingen Department of Computer Science: Database Systems*. p. 84. Archived from the original (<https://db.inf.uni-tuebingen.de/staticfiles/teaching/ss13/db2/db2-04-1up.pdf>) (PDF) on 31 October 2020.
7. Zeitler, Erik; Risch, Tore (2010). "Scalable Splitting of Massive Data Streams" (<http://urn.kb.se/resolve?urn=urn:nbn:se:uu:diva-136403>). *Database Systems for Advanced Applications*. Lecture Notes in Computer Science. Vol. 5982. pp. 184–198. doi:10.1007/978-3-642-12098-5_15 (https://doi.org/10.1007%2F978-3-642-12098-5_15). ISBN 978-3-642-12097-8.
8. Xu, Chang; Shou, Lidan; Chen, Gang; Yan, Cheng; Hu, Tianlei (2010). "Update Migration: An Efficient B+ Tree for Flash Storage". *Database Systems for Advanced Applications*. Lecture Notes in Computer Science. Vol. 5982. pp. 276–290. doi:10.1007/978-3-642-12098-5_22 (https://doi.org/10.1007%2F978-3-642-12098-5_22). ISBN 978-3-642-12097-8.
9. "ECS 165B: Database System Implementation Lecture 6" (<http://web.cs.ucdavis.edu/~green/courses/ecs165b-s10/Lecture6.pdf>) (PDF). *UC Davis CS department*. 9 April 2010. pp. 21–23.
10. Ramakrishnan, Raghu; Johannes Gehrke (2003). *Database management systems* (3rd ed.). Boston: McGraw-Hill. ISBN 0-07-246563-8. OCLC 49977005 (<https://search.worldcat.org/oclc/49977005>).
11. Raghu, Ramakrishnan; Johannes, Gehrke (2000). *Database Management Systems* (2nd ed.). McGraw-Hill Higher Education. p. 267. ISBN 978-0-07-245052-1
12. Giampaolo, Dominic (1999). *Practical File System Design with the Be File System* (<https://web.archive.org/web/20170213221835/http://www.nobius.org/~dbg/practical-file-system-design.pdf>) (PDF). Morgan Kaufmann. ISBN 1-55860-497-9. Archived from the original (<http://www.nobius.org/~dbg/practical-file-system-design.pdf>) (PDF) on 13 February 2017. Retrieved 29 July 2014.
13. "B-Trees". *Apple File System Reference* (<https://developer.apple.com/support/downloads/Apple-File-System-Reference.pdf>) (PDF). Apple Inc. 22 June 2020. p. 122. Retrieved 10 March 2021.
14. SQLite Version 3 Overview (<http://sqlite.org/version3.html>)
15. CouchDB Guide (<http://guide.couchdb.org/draft/btree.html>)

16. Tokyo Cabinet reference (<http://1978th.net/tokyocabinet/>) Archived (<https://web.archive.org/web/20090912082150/http://1978th.net/tokyocabinet/>) September 12, 2009, at the Wayback Machine
17. *Database Systems for Advanced Applications*. Japan. 2010.
18. Jagadish, H. V.; Ooi, Beng Chin; Tan, Kian-Lee; Yu, Cui; Zhang, Rui (June 2005). "iDistance: An adaptive B+-tree based indexing method for nearest neighbor search" (<http://scholarbank.nus.edu.sg/handle/10635/42315>). *ACM Transactions on Database Systems*. **30** (2): 364–397. doi:10.1145/1071610.1071612 (<https://doi.org/10.1145%2F1071610.1071612>). ISSN 0362-5915 (<https://search.worldcat.org/issn/0362-5915>). S2CID 967678 (<https://api.semanticscholar.org/CorpusID:967678>).
19. Dharamjeet; Chen, Tseng-Yi; Chang, Yuan-Hao; Wu, Chun-Feng; Lee, Chi-Heng; Shih, Wei-Kuan (December 2021). "Beyond Write-Reduction Consideration: A Wear-Leveling-Enabled B⁺-Tree Indexing Scheme Over an NVRAM-Based Architecture". *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*. **40** (12): 2455–2466. Bibcode:2021ITCAD..40.2455D (<https://ui.adsabs.harvard.edu/abs/2021ITCAD..40.2455D>). doi:10.1109/TCAD.2021.3049677 (<https://doi.org/10.1109%2FTCAD.2021.3049677>). ISSN 0278-0070 (<https://search.worldcat.org/issn/0278-0070>). S2CID 234157183 (<https://api.semanticscholar.org/CorpusID:234157183>).

External links

- B+ tree in Python, used to implement a list (<https://pypi.python.org/pypi/blist>)
 - Dr. Monge's B+ Tree index notes (<https://web.archive.org/web/20080723122307/http://www.cecs.csulb.edu/%7Emonge/classes/share/B+TreeIndexes.html>)
-

Retrieved from "https://en.wikipedia.org/w/index.php?title=B%2B_tree&oldid=1330192398"